

Perceptrons and the Perceptron Learning Algorithm

Weighted threshold units, and how they learn

Fundamentals of Deep Learning · Class 3

Prof. Nirav Bhatt

29 July

Outline

From Threshold Logic to the Perceptron

Input Space and Weight Space

The Learning Problem

The Perceptron Learning Algorithm

Convergence and Limits

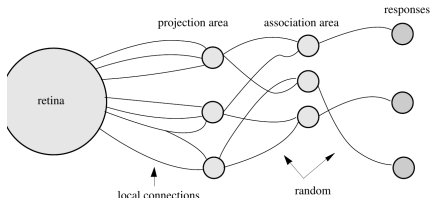
From Threshold Logic to the Perceptron

Where Class 2 left us

- A McCulloch–Pitts unit fires when $x_1 + \dots + x_n \geq \theta$ – inputs are summed with **equal weight**
- Geometrically it is a **separating plane**, but the planes are always **parallel** (unit weights, fixed orientation)
- Consequence: many linearly separable functions cannot be realised by a *single* M–P unit without extra structure

Give each input its own **weight** and add a **bias**: the plane can now take any orientation and offset. This is the **perceptron**.

A little history: Rosenblatt's perceptron

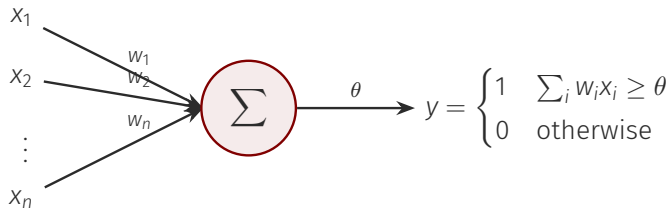


Rojas (1996), Fig. 3.1: the classical perceptron [after Rosenblatt 1958].

- Rosenblatt (1958) wired a **retina** of pixels to weighted threshold units — built to **learn** to recognise patterns
- Its only formal difference from a McCulloch–Pitts unit is the presence of **weights**
- It drew great excitement — and, later, a famous controversy over its limits

Rojas (1996), §3.1

The perceptron: a weighted threshold unit

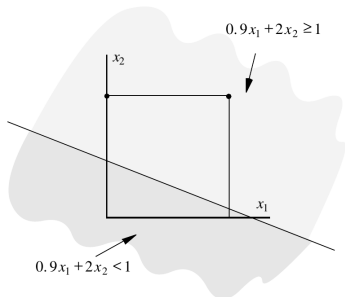


- Each input x_i carries a real **weight** w_i ; the unit fires if $\mathbf{w} \cdot \mathbf{x} \geq \theta$
- Introduced by Rosenblatt (1958); weights make the separating plane **freely orientable**

M-P fixed unit weights

PERCEPTRON **learnable** real weights

Weights tilt the separating plane



Rojas (1996), Fig. 3.4.

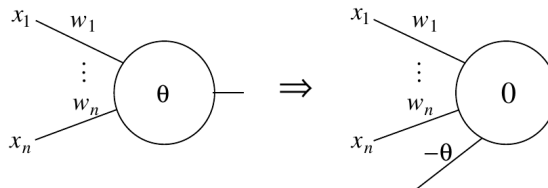
A perceptron with weights $(0.9, 2)$ and threshold 1 tests $0.9x_1 + 2x_2 \geq 1$:

- The boundary is a line of **arbitrary slope** — not just the parallel M-P planes
- Any linear separation of the plane can be realised by choosing $\mathbf{w}, \theta$

M-P parallel planes only

PERCEPTRON any orientation and offset

The bias trick



Rojas (1996), Fig. 3.5.

- Fold the threshold into a weight: add an input fixed at 1 with weight $-\theta$
- The condition $\mathbf{w} \cdot \mathbf{x} \geq \theta$ becomes $\tilde{\mathbf{w}} \cdot \tilde{\mathbf{x}} \geq 0$
- This $-\theta$ weight is the **bias**; the boundary may now miss the origin

M-P threshold θ compared explicitly

PERCEPTRON threshold absorbed as a **bias** weight

Extended input space

With the bias folded in, every input becomes an **extended vector** and the test is homogeneous:

$$\tilde{\mathbf{x}} = (x_1, \dots, x_n, 1), \quad \tilde{\mathbf{w}} = (w_1, \dots, w_n, -\theta), \quad \tilde{\mathbf{w}} \cdot \tilde{\mathbf{x}} \geq 0.$$

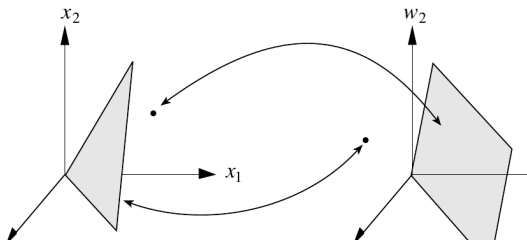
- The separating plane now passes through the **origin** of the $(n+1)$ -dimensional space
- Training data split into two sets: P (should output 1) and N (should output 0)
- Goal: find $\tilde{\mathbf{w}}$ with $\tilde{\mathbf{w}} \cdot \mathbf{x} > 0$ for all $\mathbf{x} \in P$ and < 0 for all $\mathbf{x} \in N$

M-P hand-designed weights

PERCEPTRON weights to be **found** from labelled data

Input Space and Weight Space

Two dual viewpoints



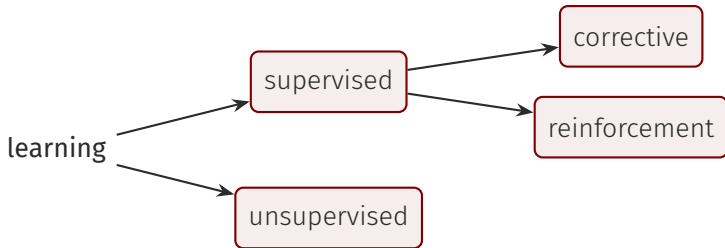
Rojas (1996), Fig. 3.7.

- In **input space**, a fixed $\mathbf{w}$ is a plane splitting the inputs
- In **weight space**, a fixed input $\mathbf{x}$ is a plane splitting the weight vectors
- $\mathbf{w} \cdot \mathbf{x} = 0$ is symmetric: points and planes **swap roles**

Learning is a search in **weight space** for a point on the correct side of every data plane.

The Learning Problem

What does it mean to 'learn'?



- **Supervised**: each input comes with a target; **unsupervised**: the network finds structure on its own
- Supervised splits into **corrective** (fix the error directly) and **reinforcement** (reward or punish)
- The **perceptron** rule is supervised with error correction

Rojas (1996), §4.1.1

Learning as a search for parameters

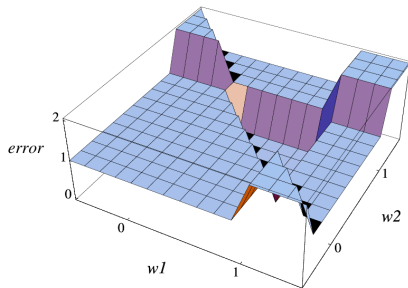
- A network is a **parametric system**: fix the architecture, and the weights are **free parameters** to be set
- “Learning” means **searching** weight space for values that make the outputs match the targets
- The search is driven by an **error signal**: how wrong the current weights are on the training data
- Every learning rule in this course is a different way of **descending** that error

Rojas (1996), §4.1

M-P weights fixed by the designer

PERCEPTRON weights **searched** to minimise error

The error surface



Rojas (1996), Fig. 4.5: error for AND, over weights

(w_1, w_2) .

- Each point is a weight vector; the height is **how many examples it misclassifies**
- The flat **valley floor** (error 0) is the **solution region**
- Learning = walking downhill to that valley, using only **local** decisions

M-P no error to descend — weights hand-set

PERCEPTRON descend the error surface to a zero-error region

Absolute linear separability

- The PLA needs a **strict** separation: every positive example on the *open* positive side, every negative on the open negative side
 - If a separating plane exists, we can always find one with a **margin** — no data point lies exactly on it
 - This **absolute** separability is what the convergence proof will rely on
- (1996), §4.1.3

Rojas

“Separable” is not quite enough — we need a little room on each side. Then learning is guaranteed to finish.

The Perceptron Learning Algorithm

Learning as error correction

- Start from a random weight vector; present training examples one at a time
- If the current example is classified **correctly**, leave the weights unchanged
- If it is **wrong**, nudge the weights toward the correct decision — and repeat

Only mistakes drive learning. The rule is local, online, and strikingly simple.

The algorithm

Sets P (target 1) and N (target 0), extended vectors. Seek $\mathbf{w}$ with $\mathbf{w} \cdot \mathbf{x} > 0$ on P , $\mathbf{w} \cdot \mathbf{x} \leq 0$ on N (Rojas, 1996, §4.2):

start: random $\mathbf{w}_0$, $t := 0$

test: pick $\mathbf{x} \in P \cup N$ at random

if correctly classified: go to **test**

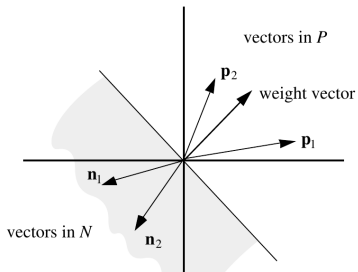
if $\mathbf{x} \in P$ and $\mathbf{w}_t \cdot \mathbf{x} \leq 0$: go to **add**

if $\mathbf{x} \in N$ and $\mathbf{w}_t \cdot \mathbf{x} \geq 0$: go to **subtract**

add: $\mathbf{w}_{t+1} = \mathbf{w}_t + \mathbf{x}$, $t := t + 1$; go to **test**

subtract: $\mathbf{w}_{t+1} = \mathbf{w}_t - \mathbf{x}$, $t := t + 1$; go to **test**

Why the update works



Rojas (1996), Fig. 4.8.

A misclassified $\mathbf{x} \in P$ has $\mathbf{w} \cdot \mathbf{x} \leq 0$. After $\mathbf{w}' = \mathbf{w} + \mathbf{x}$:

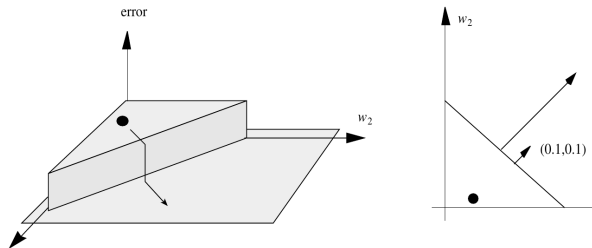
$$\mathbf{w}' \cdot \mathbf{x} = \mathbf{w} \cdot \mathbf{x} + \|\mathbf{x}\|^2$$

- The dot product **increases** — the weight rotates *toward* $\mathbf{x}$
- For $\mathbf{x} \in N$ we subtract, rotating *away* — symmetric fix

M-P no update rule — weights set by hand

PERCEPTRON each mistake rotates $\mathbf{w}$ the right way

The update on the error surface



Rojas (1996), Fig. 4.11: error surface (left) and weight space (right).

- Each correction moves $\mathbf{w}$ into a region of **lower error**
- In weight space, every misclassified point carves off a half-space; the solution is their **intersection**
- The algorithm walks toward that **solution region**

Fixing the error in one step

The plain rule nudges; we can instead **fully correct** a mistake. For $\mathbf{x} \in P$ with $\mathbf{w} \cdot \mathbf{x} = -\delta \leq 0$, set (Rojas, 1996, §4.2.3):

$$\mathbf{w}' = \mathbf{w} + \frac{\delta + \varepsilon}{\|\mathbf{x}\|^2} \mathbf{x} \quad \Rightarrow \quad \mathbf{w}' \cdot \mathbf{x} = \varepsilon > 0.$$

- One update moves $\mathbf{x}$ just past the boundary (by a margin ε) — the mistake is **fixed immediately**
- Making ε small avoids overshooting into a worse region
- This **corrective** variant learns faster than the plain add/subtract rule

M-P no notion of correction

PERCEPTRON each step **eliminates** one error outright

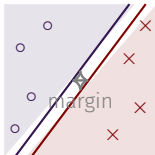
Convergence and Limits

The perceptron convergence theorem

If P and N are **linearly separable**, the perceptron learning algorithm makes only a **finite** number of weight updates — it converges to a separating $\mathbf{w}$ in finite time.

- Idea of the proof: if a solution $\mathbf{w}^*$ exists, each update increases the alignment $\mathbf{w} \cdot \mathbf{w}^*$ faster than it grows $\|\mathbf{w}\|$
- Bounding the two quantities forces the number of updates to be finite Rojas (1996), §4.2–4.3
- Guarantee of **existence and termination** — not of speed

How long can it take? Worst case



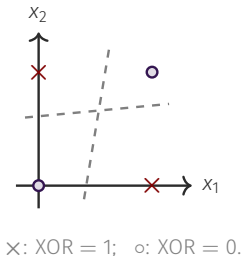
cf. Rojas (1996), §4.2.5.

- Convergence is guaranteed, but the step count can be **large**
- A **thin margin** means the weight vector must be nudged many times — updates grow with the **inverse squared margin**
- Still **finite**: the theorem holds, only the speed suffers

M-P no learning — nothing to time

PERCEPTRON finite, but slow when the margin is thin

The limit: XOR and non-separable data



- A single perceptron computes exactly the **linearly separable** functions — any orientation, but still one line
- **XOR** is not separable: no line puts both $\times$ on one side and both $\circ$ on the other
- On non-separable data the PLA **never terminates**; the **pocket algorithm** keeps the best weights seen

Rojas, §4.2.4

One layer is still a linear classifier. XOR needs a **hidden layer** — the shallow network (Classes 5–7).

Takeaways

1. A **perceptron** is a threshold unit with **learnable weights** and a **bias** — its boundary is a freely oriented hyperplane
2. The bias trick makes the test homogeneous: $\tilde{\mathbf{w}} \cdot \tilde{\mathbf{x}} \geq 0$
3. The **PLA** corrects only on mistakes: add $\mathbf{x}$ for a missed positive, subtract for a missed negative
4. It **provably converges** when the data are linearly separable — but XOR and non-separable sets remain out of reach

Class 5: stack weighted units into layers — the shallow neural network.

References

- R. Rojas. *Neural Networks: A Systematic Introduction*. Springer, 1996. Chapters 3–4. Figures 3.4, 3.5, 3.7, 4.8, 4.11 reproduced with attribution.
- F. Rosenblatt (1958). “The perceptron: a probabilistic model for information storage and organization in the brain.” *Psychological Review*, 65(6), 386–408.
- NPTEL Computational Neuroscience notes (perceptrons and MLPs), used for supporting intuition.

Questions?